

ComplexGitSync 0000.04– User Guide

Nicolas Flipó

August 25, 2026

Contents

1	User Guide	2
2	CLI Overview	2
3	Lifecycle Contract	2
4	Commands	2
4.1	initialise	2
4.2	bootstrap	3
4.3	configure	4
4.4	create-cgs	4
4.5	clean-init	4
4.6	purge	4
4.7	pull	4
4.8	pull-force	5
4.9	clone	5
4.10	checkout	5
4.11	add	5
4.12	commit	6
4.13	push	6
4.14	freeze-release	6
4.15	freeze	6
4.16	launch-release	7
4.17	view-tree	7
4.18	status	7
4.19	remember	7
4.20	memorize	7
4.21	retrieve	8
4.22	reload	8
5	Document Formats	8
5.1	Local Authoring Spec (.cgs)	8
5.2	Repository Placement and Nested Discovery	9
5.3	Git Tree State Snapshot (.gts)	10
5.4	Planning / GOC Documents (.goc)	11
5.5	Local Git Register and Sync Ledger (.lgr)	11
6	Configuration and Environment	11
7	Error Reference	12

1 User Guide

This chapter is the authoritative reference for every user-facing command, configuration option, document format, and error condition exposed by `ComplexGitSync`.

2 CLI Overview

The single entry-point is the `cgitsync` command.

```
$ cgitsync <command> [options]
```

Running `cgitsync` without a command prints a help summary and exits with code 0.

3 Lifecycle Contract

This guide follows the simplified canonical lifecycle:

1. `initialise(.cgs)` → clone all repos → **READY** (*new project*)
`initialise(.gts)` → restore from snapshot → **READY** (*existing project*)
2. `pull(.cgs|.gts)` → resync an existing tree
3. `checkout(.gts)` / `add` / `commit` / `push` / `tag` → git operations on the tree
4. `freeze` → emit the next `.gts` id and update `<Project_name>.lgr`

The project-local `.lgr` register and sync ledger are both active (T24, T32). Every synchronisation operation is automatically recorded as an immutable DAG event in the `[[ledger]]` section of the `.lgr` file.

4 Commands

The CLI exposes four command groups once a `GitTree` is **READY**. The minimalist level is `initialise`, `bootstrap`, `clean-init`, `freeze-release`, `freeze-release-force`, `status`, `view-tree`, and `launch-release`. The expert level exposes the individual operations: `branch`, `checkout`, `purge`, `validate`, `clone`, `pull`, `pull-force`, `add`, `commit`, `push`, `tag`, and `freeze`. The configuration level, `configure` and `create-cgs`, authors a `.cgs` specification without requiring existing state. The memory level, `remember`, `memorize`, `retrieve`, and `reload`, binds a project to an external SSH-Git *Memory* repository (distinct from the project's own remotes) and persists or restores a finalized local Memory State. Redundant inspection aliases such as `print`, `view-operation`, and `validate-topology` are not part of the public CLI surface.

4.1 initialise

```
$ cgitsync initialise <spec.cgs> [--output-path DIR] [--commit-gitignore
]
    [--force-gitignore-sync] [--git-user-name NAME] [--git-user-email
    EMAIL]
$ cgitsync initialise <snapshot.gts>
$ cgitsync initialise --project NAME --repo PROVIDER:OWNER/REPO [--repo
    ...] [--output-path DIR]
```

Unified initialisation entry point. Dispatches on file extension:

- **.cgs** source (clone mode): clones the full repository tree. The optional `-output-path` controls `CGSPATH`; `CGSHOME` is derived as `CGSPATH/<project-name>`. When omitted, `CGSPATH` defaults to `../...`. On success, ends in `READY`. If clone setup fails because stale generated state is still present, the CLI prints `Try clean-init method`.
- **.gts** source (restore mode): loads a saved snapshot directly. On success, ends in the lifecycle state recorded in the snapshot.
- Direct CLI authoring: requires `-project` and at least one repeatable `-repo`. It is mutually exclusive with a source file. The CLI collects values only and calls `ComplexGitSyncClient.configure()`; that reusable Python API delegates parsing, normalization, and validation to `cgs_format.py` and produces the same canonical `CgsDocument` as an equivalent `.cgs` file.

All initialization paths end in a `READY` tree or fail explicitly.

Before a `.cgs` source is confirmed ready, every repository with children (root, or any nested repository that itself has further nested children) is safely pulled parent-first and has its `.gitignore` updated with the relative path of each immediate child — nested repositories are plain independent clones rather than gitlinks, so without this a parent’s `git status/git add` would otherwise see a child’s working tree as ordinary untracked content. This is logged as the `GT-GITIGNORE` phase, after `GT-CLONE`. If the safe pull for one of these repositories fails, `initialise` raises immediately and nothing is written; pass `-force-gitignore-sync` to instead fall back to a pull-force recovery for that one repository (never a force-push). By default nothing is staged, committed, or pushed by this step — it only writes the file and prints what changed. Pass `-commit-gitignore` to explicitly approve staging (`.gitignore` alone), committing (with a message listing exactly which children were added), and pushing each changed repository. The commit identity defaults to local git config; `-git-user-name/-git-user-email` override it and persist the override to `CGSHOME/.cgitsync/master.toml` via `MasterConfig`, so later invocations on the same workspace pick it up without repeating the flags.

4.2 bootstrap

```
$ cgitsync bootstrap <spec.cgs> <project_name> [--cgs-path DIR]
```

Clones the full repository tree from a `.cgs` source into an isolated `CGSHOME`, for running `cgitsync` from its own standalone clone rather than nested inside the project tree it manages — one `ComplexGitSync` clone reused across many projects. It delegates to the same `ComplexGitSyncClient.clone_cgs` pipeline as `clone` below, and so shares its branch-fallback and nested-discovery behaviour, but differs from both `clone` and `initialise` in two ways:

- `project_name` is a required positional argument, not inferred from the `.cgs` document. It always forms the final path segment of `CGSHOME`, regardless of the specification’s own project name.
- `CGSPATH` does not default to `../..` relative to the current directory (that default assumes `ComplexGitSync` is cloned inside the tree it manages — the nested-mode case `initialise` is for). When `-cgs-path` is omitted, `CGSPATH` instead defaults to a fresh `$HOME/.cgs/CGS<timestamp>/` directory, created automatically if it does not exist, so a bootstrapped project can never land inside `ComplexGitSync`’s own clone.

4.3 configure

```
$ cgitSync configure [--output FILE]
```

Interactively prompts for a project name, default branch, and one or more repositories (provider, owner, name, and per-repository overrides), then writes the resulting `.cgs` file through the same offline `ComplexGitSyncClient.configure()` facade used by `initialise -project/-repo` and `create-cgs`. The provider prompt accepts `github`, `gitlab`, `codeberg`, or `custom`; `custom` additionally prompts for the provider URL. When `-output` is omitted, the CLI prompts for a destination path, defaulting to `<project-name>.cgs`.

4.4 create-cgs

```
$ cgitSync create-cgs --project CGSil1 \  
  --repo github:flipoyo/ComplexGitSync \  
  --repo codeberg:GX4G/GX4G \  
  --output CGSil1.cgs
```

Builds the canonical `CgsDocument` through the offline format pipeline and serializes concise `.cgs` TOML. It performs no Git or network work.

4.5 clean-init

```
$ cgitSync clean-init <spec.cgs> [--output-path DIR] [--commit-gitignore  
  ]  
  [--force-gitignore-sync] [--git-user-name NAME] [--git-user-email  
    EMAIL]
```

Recovery-oriented initialisation for a `.cgs` source. It follows the same high-level pipeline as `initialise(.cgs)` (including the `.gitignore` lifecycle sync described there), but inserts a cleanup phase before cloning:

`load->expand->validate->purge->clone->gitignore`

Use it when an earlier initialisation attempt left partial child checkouts. `CGSHOME/.cgitsync/master.toml` (the persisted Git identity override, if any) is preserved by the purge phase — it is workspace-local configuration, not generated clone state.

4.6 purge

```
$ cgitSync purge <spec.cgs> [--output-path DIR]
```

Removes generated clone state from the resolved `CGSHOME` without cloning. It deletes child repository directories declared directly under the project root and project-local `*.lgr` files. Nested directories that are not immediate root children, and `.cgitsync/master.toml` (see `clean-init` above), are left untouched by this command.

4.7 pull

```
$ cgitSync pull [spec.cgs|snapshot.gts] [--commit-gitignore]  
  [--force-gitignore-sync] [--git-user-name NAME] [--git-user-email  
    EMAIL]
```

Resynchronises the project tree. `.cgs` sources reload the source, discover nested configs, then resynchronise the loaded tree. `.gts` sources load the saved registry as the starting point and then run the same pull workflow. In both cases the operation starts at the project root: `root -> parent -> leaf`. Every repository — root, parent, and leaf alike — is a plain independent clone and receives its own `git pull -ff-only`. Use `launch-release` when you want to check out a frozen recorded release state instead of pulling current remotes. If local changes block this safe pull, the CLI prints `You can try cgitssync pull-force command`.

For a `.cgs` source, `pull` also runs the same `.gitignore` lifecycle sync described under `initialise` — including `-commit-gitignore`, `-force-gitignore-sync`, and the `-git-user-name/-git-user` identity override — once the tree-wide pull completes. A `.gts` source runs no discovery, so there is nothing new for the sync to find. `pull-force` does not run this sync at all; it is a destructive recovery command, not a lifecycle path the sync is wired into.

4.8 pull-force

```
$ cgitssync pull-force [spec.cgs|snapshot.gts]
```

Destructively resynchronises the same tree. Every repository — root, parent, and leaf alike — is forced to the selected remote branch, parent-first, with `git fetch`, `git checkout -B <branch> FETCH_HEAD`, and `git clean -fd`. This command can discard local uncommitted changes and untracked files; it never force-pushes.

4.9 clone

```
$ cgitssync clone <spec.cgs> [--target-dir DIR] [--output-path DIR]
```

Clones the full repository tree declared by the `.cgs` file. This CLI command delegates to `ComplexGitSyncClient.clone`. When `-target-dir` is omitted, the project root is derived from the specification. `-output-path` provides the parent directory used to derive the project root.

For each repository, `cgitssync` tries the repo target branch first and falls back to the repo fallback branch when the preferred branch does not exist on the remote. Nested `.cgs` files are discovered after parent repositories are cloned, so explicit and automatic nested trees are materialised during the same command.

4.10 checkout

```
$ cgitssync checkout <branch> [--gts <snapshot.gts>] [--ref-kind branch|tag]
```

Checks out a branch or tag across the whole tree, parent-first. Loads the `READY` registry from `-gts`, then propagates the ref, creates missing local branches, and runs `git checkout` in every repo. On success the tree remains `READY` and a new `.gts` snapshot is written.

4.11 add

```
$ cgitssync add [--gts <snapshot.gts>] [--dry-run]
```

Stages all changes across the tree (`git add -all`), leaf-first. Requires `READY`; tree stays `READY`. With `-dry-run`, `cgitssync` prints the leaf-first execution plan and does not mutate repositories.

4.12 commit

```
$ cgitasync commit <message> [--gts <snapshot.gts>] [--no-stage] [--dry-run]
```

Commits changes across the tree, leaf-first. Loads the READY registry from `-gts`. Repositories with no staged changes after the optional `git add -all` step (suppressed by `-no-stage`) are silently skipped. Requires a READY tree; the tree remains READY on success. Commit preflight now emits warnings for actionable workspace issues such as dirty worktrees, ahead branches, or stale recorded `commit_sha` values. With `-dry-run`, `cgitasync` prints the planned stage/commit sequence without performing git writes.

The commit message conventionally ends with `CGS#VERSION`.

4.13 push

```
$ cgitasync push [--gts <snapshot.gts>] [--dry-run]
```

Pushes all repositories to their configured remotes, leaf-first. Loads the READY registry from `-gts`. Each repo is pushed to `entry.remote_name` (defaulting to "origin") on its currently resolved branch. Refreshes the stored hash in `GitTree`. Requires a READY tree; the tree remains READY on success. Push preflight blocks detached HEADs, missing remotes, unresolved merges, and behind/diverged branches; it warns when the local branch is ahead or when the loaded snapshot records an older `commit_sha`. With `-dry-run`, `cgitasync` prints the push plan without mutating any repository.

4.14 freeze-release

```
$ cgitasync freeze-release <name> <message> [--gts <snapshot.gts>] [--dry-run]
$ cgitasync freeze-release-force <name> <message> [--gts <snapshot.gts>] [--dry-run]
```

Minimalist release workflow from a READY tree. It chains public operations in order: `add -> commit -> pull -> push -> freeze`. The force variant uses `pull-force` instead of `pull`. The release name becomes the shared tag and freeze snapshot label; the message is used for the release commit.

4.15 freeze

```
$ cgitasync freeze <name> [--gts <snapshot.gts>] [--dry-run]
```

Performs a release freeze across all repos, leaf-first: stage/commit, create+push the shared release tag, refresh SHA/state, and write a `.gts` snapshot. Loads the READY registry from `-gts`. Requires READY and leaves the tree READY. Freeze preflight checks remote availability, branch alignment, detached HEADs, unresolved merges, and tag absence. Dirty worktrees and stale recorded `commit_sha` values are reported as warnings because freeze will stage, commit, and snapshot them. Freeze snapshots also include a deterministic `freeze_manifest` section that records freeze invariants (immutable snapshot, validated workspace, synchronized tag, `release-name`, ledger checkpoint) and declares `launch_state` as the canonical restore operation. Their immutable snapshot filename includes the release, for example `gts-000001-release-2026.05.gts`, while the ledger id remains `gts-000001`. With `-dry-run`, `cgitasync` prints the planned add/-commit/tag/push graph for the loaded workspace without mutating it.

4.16 launch-release

```
$ cgitasync launch-release <name> [--gts <snapshot.gts>]
```

Checks out the frozen release tag `<name>` across the loaded READY tree, parent-first, and writes a fresh `.gts` snapshot. This is the release oriented CLI path for returning a workspace to a frozen version; direct tag creation remains available from the Python client API as `tag()`.

The integration suite validates local no-network lifecycle coverage through the CLI path, including initialisation and the READY git command cycle `add -> commit -> push -> freeze -> launch-release`.

4.17 view-tree

```
$ cgitasync view-tree [spec.cgs|snapshot.gts] [--depth N] [--collapse REPO]
```

Renders a deterministic terminal tree that focuses on repository topology and operational state: `name [branch] local_state sync_state`. Use `-depth` to limit visible levels and repeat `-collapse` to hide selected subtrees.

Branch Topology Propagation Rules (T35):

1. **Reference branch:** The root repository's current branch is the canonical reference for all repos.
2. **Leaf-to-root inheritance:** Branch targeting flows root-first via `propagate_global_branch`. `validate-topology` verifies the on-disk state without issuing any git writes.
3. **Allowed divergence:** Repositories in a frozen (TAG) state produce a `tag_divergence` diagnostic that does *not* make the topology incoherent.
4. **Incoherent states (blocking):** `misaligned_branch` (different branch from root) and `detached_head` (detached HEAD without a tag reference).

4.18 status

```
$ cgitasync status [--gts FILE]
```

Summarises tree readiness and per-repository sync state. The table reports the local branch, upstream branch, local worktree state, ahead/behind tracking counts, current HEAD, and the commit recorded in the loaded `.gts`.

4.19 remember

```
$ cgitasync remember <spec.cgs> [--output-path DIR] [--service HOST] [--remote-name NAME]
```

Binds the given `.cgs` artefact to its external SSH-Git Memory endpoint, resolving `CGSHOME` the same way `initialise` does. `-service` defaults to `forge43.io`; `-remote-name` defaults to `forge43`.

4.20 memorize

```
$ cgitasync memorize <memory-state-dir> [--branch BRANCH]
```

Persists a finalized local Memory State directory (`.cgitasync/state(<hash>)_i/`) to the configured SSH-Git Memory remote, on `-branch` (default `main`).

4.21 retrieve

```
$ cgitsync retrieve <name> [--output-path DIR] [--branch BRANCH] [--  
service HOST] [--remote-name NAME]
```

Retrieves the named external SSH-Git Memory repository into a clean CGSHOME (CGSPATH/<name>; defaults to \$CGSHOME or the current directory when -output-path is omitted).

4.22 reload

```
$ cgitsync reload <name> [--output-path DIR] [--branch BRANCH] [--  
service HOST] [--remote-name NAME]
```

Retrieves the named external Memory repository, then restores the ComplexGitSync execution context from it — the recovery path for a CGSHOME that only has its Memory binding left.

5 Document Formats

.cgs means **ComplexGitSync** specification, .gts means **GitTreeState** snapshot, .goc means **GitOrchestrationCommand** plan, and .lgr means **Local Git Register**.

5.1 Local Authoring Spec (.cgs)

A TOML file that describes the repository tree to manage. It is the only hand-authored input; it is never modified at runtime. The normal form has two top-level keys:

```
project = "CGSi11"  
repos = [  
  "gitlab:CGS_test/CGSi11",  
  "gitlab:CGS_test/CGSi12",  
  "github:flipoyo/CGSi11",  
]
```

Repository identifiers have the form **provider:owner/repository**. Nested GitLab namespaces are supported; the final path segment is the repository name. The document pipeline is:

PARSE -> NORMALIZE -> VALIDATE -> canonical CgsDocument

The canonical providers and hosts are:

Provider	Host	Remote generation
github	github.com	SSH and HTTPS
gitlab	gitlab.com	SSH and HTTPS
codeberg	codeberg.org	SSH and HTTPS
custom	explicit URL required	SSH and HTTPS

Repository-ID parsing is provider-independent and deterministic. For example:

```
codeberg:GX4G/GX4G
```

normalizes to `gitprovider = codeberg`, `project_owner_name = GX4G`, and `repo_name = GX4G`. A custom identifier must use an inline table so the host is explicit:

```
{ repository = "custom:team/tool", gitprovider_url = "https://git.  
example.com" }
```


No host is inferred for `custom`. The configured access protocol then selects `git@host:owner/repository.git` or `https://host/owner/repository.git`.

The textual grammar and its only parser, `parse_repo_id()`, live in `cgs_format.py`. The provider model in `git_repo.py` receives the already-separated provider, owner, and repository fields and composes remotes; it does not parse authoring shorthand.

The `.cgs` format pipeline is deterministic and offline. A syntactically valid repository may therefore reference a host, owner, branch, or tag that is not currently reachable. Remote existence and reference availability are checked only by explicit Git operations during runtime resolution and execution.

Normalization supplies `default_branch = main`, makes `fallback_branch` equal to that default, selects `access_protocol = ssh`, enables `nested_config = auto`, and uses the repository name as `relative_path`. If exactly one repository name matches `project`, its path is inferred as `..`.

Advanced inline tables are used only where a value differs from those defaults:

```
project = "demo"
repos = [
  "github:owner/demo",
  { repository = "github:owner/docs", relative_path = "documentation",
    nested_config = "disabled" },
]
```

The legacy advanced `[project]` and `[[repos]]` tables remain accepted for project-wide branch settings, custom provider URLs, tags, and other explicit overrides. Unknown providers, malformed identifiers, duplicates, and missing `project` or `repos` are rejected before a `GitTree` is built.

Two checked-in files document this boundary. Copy `examples/template.cgs` for normal authoring. Its equivalent `examples/normalized_template.cgs` expands every inferred field and is provided as a developer reference for the canonical `CgsDocument` shape. The two files are regression-tested for semantic equivalence.

Generated specifications use the same concise representation. The `GitTree.to_cgs()` convenience method delegates model projection to `cgs_format.py`; neither `GitTree` nor `GitRepo` formats TOML or implements the authoring grammar. Serialization omits every default that normalization can deterministically reconstruct and retains inline tables only for exceptional settings such as a non-default branch, path, protocol, tag, custom host, or discovery rule.

The supported round trip is semantic:

```
.cgs -> CgsDocument -> GitTree -> CgsDocument -> .cgs
```

Parsing and normalizing the generated file must reproduce the initial canonical document. Whitespace, comments, and TOML table layout need not be byte-for-byte identical.

5.2 Repository Placement and Nested Discovery

For every non-root entry, `relative_path` is relative to the parent repository represented by the current `.cgs`; it is not relative to the process working directory. At the top level that parent is the project root. While expanding a nested config, it is the repository containing that config. The default is the repository name. A unique repository matching the project name uses `.` because it represents the current root.

The `nested_config` option controls whether traversal continues after the repository is available:

- `auto` (default) loads the sole root-level `*.cgs` file, if one exists; multiple matches are rejected as ambiguous.

- `disabled` does not inspect the repository for nested config.
- A relative path ending in `.cgs` loads that exact file inside the repository. It may not escape the repository root.

For the integration topology, `CGSil2.cgs` contains:

```
{ repository = "github:flipoyo/CGSih1", relative_path = "../CGSih1",
  nested_config = "disabled" }
```

Because the config describes the tree below `CGSil2`, the path resolves from that directory:

`<CGSHOME>/CGSil2/../CGSih1 -> <CGSHOME>/CGSih1`

It therefore references the existing sibling declared by `CGSil1.cgs`, instead of cloning another `CGSih1` under `CGSil2`. Discovery recognizes the already-registered absolute path and keeps the canonical entry. The cross-reference is `disabled` because the canonical root-level `CGSih1` entry is responsible for loading `CGSih1.cgs`.

5.3 Git Tree State Snapshot (`.gts`)

A TOML file generated automatically by bootstrapping and release operations. It must never be hand-edited. It captures the exact commit SHAs and lifecycle state of the tree at a point in time.

Top-level sections:

- `[document]` — `format_version`, `schema_version`, `generated_at`, `command_origin`, `hash_algorithm`, `snapshot_hash`.
- `[project]` — project name and `root_absolute_path`.
- `[tree_state]` — `lifecycle_state`, `is_ready`, `registry_complete`.
- `[[repo_state]]` — one entry per repo; includes `commit_sha`, `sync_state`, and ref information.

Why both version fields and hash metadata exist:

- `format_version` identifies the broad document family and keeps long-range compatibility intent explicit.
- `schema_version` identifies the exact field-level contract used by validation and canonical snapshot serialization.
- `snapshot_hash` (with `hash_algorithm = "sha256"`) provides deterministic workspace identity: identical tree state produces identical digest even if volatile metadata (`generated_at`, `command_origin`) differs.

How this is used at runtime:

- During snapshot generation, `ComplexGitSync` writes `schema_version`, `hash_algorithm`, and canonical `snapshot_hash`.
- During load/validation, `snapshot_hash` is recomputed from the canonical payload and must match.
- The project-local `.lgr` register uses this canonical hash to deduplicate equivalent snapshots.

5.4 Planning / GOC Documents (.goc)

Used for project identity and planning metadata. Supported in TOML, YAML, or JSON. YAML support requires the optional PyYAML dependency.

5.5 Local Git Register and Sync Ledger (.lgr)

A TOML file maintained per project at `<project-root>/<Project_name>.lgr`. It is updated automatically whenever a `.gts` snapshot is written.

Sections:

- `[register]` — current snapshot pointer (`current_snapshot_id`, `current_snapshot_hash`, `current_snapshot_path`).
- `[[snapshots]]` — list of stable `gts-XXXXXX` identifiers, one per unique workspace state, deduplicated by SHA-256 hash.
- `[[ledger]]` — append-only list of synchronisation events forming a DAG via `parent_sync_ids`.

Ledger event schema:

```
[[ledger]]
sync_id      = "lgr-000001"
parent_sync_ids = []
operation    = "clone"
timestamp    = "2026-05-20T19:48:50.159Z"
actor        = "username"
workspace_hash = "5f7c8a2d..." % 64-character SHA-256 hex digest
gts_snapshot_id = "gts-000001"
affected_repos = ["demo", "dep-a"]
```

The `workspace_hash` field equals the canonical SHA-256 digest (`document.snapshot_hash`) of the associated `.gts` file, creating a direct link between each ledger event and the immutable workspace state it records. Events are parents-before-children in DAG order.

6 Configuration and Environment

`ComplexGitSync` uses project files for topology and does not require global topology configuration. Snapshot discovery can use `CGSHOME`; when it is not set, the `initialise(.cgs)` output-path default is `CGSPATH=../..` (nested mode: `ComplexGitSync` cloned inside the tree it manages); later commands can also walk upward from the current directory to find `.cgitsync/state/`. `bootstrap(.cgs)` uses a different default instead — `CGSPATH=$HOME/.cgs/CGS<timestamp>/`, created automatically — since it is meant for a `ComplexGitSync` clone that is not nested inside any project tree; see `bootstrap` above. The logging and runtime-state subsystems use the user state directory by default:

- logs: `/.local/state/ComplexGitSync/logs/`
- latest runtime-state mapping: `/.local/state/ComplexGitSync/runtime/`

If `XDG_STATE_HOME` is set, these directories are rooted there instead.

7 Error Reference

ConfigValidationError Raised when `project` or `repos` is missing, a repository identifier is malformed, or a canonical field is invalid (for example an unknown `gitprovider`).

GitSyncError Raised when clone-time Git operations fail, for example because the remote branch cannot be resolved or the target directory is already occupied.

Exit code 1 General CLI failure (parse error, validation error).

Exit code 2 Command exists but is not yet implemented.

8 Troubleshooting

“gitprovider_url is required for custom provider” Add a `gitprovider_url` field to the offending `[[repos]]` entry in your `.cgs` file.

validate exits non-zero despite a correct-looking file Check that `project` is present and every `repos` entry uses `provider:owner/repository`. For advanced `[[repos]]` blocks without a `repository` shorthand, include `project_owner_name` and `project_name`.

Clone destination already exists and is not empty Re-run `clone` with `-target-dir` to force a clean location, or let `cgitsync` choose a suffixed default directory.

No cloneable branch found for <repo> Confirm that the remote exposes either the repo’s `default_branch` or its `fallback_branch`.

PyYAML import error when using .goc files Add PyYAML in the Pixi environment: `pixi add pyyaml`.